Project brief and rules for the AI coding assistant

Team: Code Blooded. Event: Smart India Hackathon, Problem Statement 26265 ("Streamline of security deposit/ FDR portal", AICTE Regulation Bureau, Smart Automation, Software). Working product name: NidhiNetra (not final).

Read this whole file before writing code. It is the project memory. When it conflicts with a guess you would otherwise make, follow this file. When something is not covered here, ask instead of inventing.

---

1. How to behave

- Act as a senior fintech/GovTech engineer building a prototype that must look and behave like a real, scalable system.
- Prefer small, working, tested increments. Show the plan in a few lines, then implement.
- Never invent AICTE rules, amounts, clause numbers, bank APIs or statistics. If a value is unknown, put it in configuration with a clear `TODO(confirm)` and a placeholder.
- Keep claims modest. Anything simulated must be labelled "simulated" in code, UI and docs.
- Ask a question when a decision changes the data model, the state machine or the audit log. Otherwise choose the simplest option and say what you chose.

2. The project in one paragraph

AICTE needs a reliable way to know which required security deposits / FDRs exist, whether their details and any lien have been checked, when they mature, what needs renewal, and whether release requests have completed the required checks. Today this runs on PDFs, emails, physical files and Excel. We are building one secure portal with: an institution digital vault, bank verification (simulated), a maturity and renewal alert engine, role-based AICTE queues and dashboard, a guarded release/clearance workflow, and a tamper-evident audit trail. The core idea: keep what an institution SUBMITS separate from what a bank CONFIRMS, and never advance a case on the institution's word alone.

3. History: how we got here (so you understand the choices)

1. We started from the official SIH 26265 text and asked for a deep research report on the FDR-with-lien lifecycle, market solutions, tech stacks, and references.
2. We compared three AI research reports (Claude, Gemini, ChatGPT) and rated their quality. Gemini had the most AICTE-specific lifecycle detail but made unverified claims (bank lien APIs, penalty percentages, ID-document accuracy figures applied to FDRs). Claude's report was the best sourced and flagged what was unverified. ChatGPT's was short and generic.
3. We resolved their disagreements and wrote an implementation blueprint (problem, requirements R1 to R10, gaps, USPs, architecture, plan, risks).
4. We rewrote the six-slide PPT content to remove unmeasured numbers and overclaims.
5. The team wrote a plain-English problem explainer (adds: Trust/Society entity, "source unavailable" handling, "never auto-release", "a flag is not proof").

Key findings that drive the design:

- No public API lets a third party verify an arbitrary FDR and its lien. The Account Aggregator term-deposit schema has no lien field and AICTE cannot be a Financial Information User. So the BANK must be the authority on lien status, and our bank connection is a pluggable adapter that is SIMULATED in the prototype.
- The closest real patterns: NeSL e-Bank Guarantee (bank creates the record; beneficiary gets notify, pull and event APIs; status closes only when the bank acts) and clearing-corporation e-FDR (bank marks lien; release by API).
- A PDF carries no proof it came from the bank. Renewal changes FDR number and dates, so lien continuity needs a lineage, not one row per FDR number.
- Deposit tenure varies by handbook edition (8 years in an older letter of intent, 10 years in 2024-27 guidelines). Penalty rules are reported differently by different sources. Everything rule-like lives in configuration.
- We chose a hash-chained audit log, not blockchain, and OCR only to pre-fill fields, not to judge authenticity.

4. Source of truth and docs

Keep these in `/docs` and read them when relevant:

1. Official SIH 26265 problem statement (ground truth for scope).
2. `Code_Blooded_SIH26265_Crisp_Problem_Explainer.pdf` (team's plain-English briefing).
3. `AICTE_FDR_portal_implementation_blueprint.pdf` (our decisions, architecture, plan).
4. `AICTE_FDR_Portal_Architecture_Research.pdf` (Gemini report: useful for AICTE lifecycle and business rules; treat its API and penalty claims as UNVERIFIED).
5. The earlier Claude research report (best sourced).

If documents disagree, the order above wins, except that handbook rules must be confirmed against the official AICTE Approval Process Handbook.

5. Non-negotiable rules

1. Do not hardcode amounts (for example 15 to 35 lakh), tenure (for example 10 years), penalty percentages, reminder days or escalation days. Store them in a `rules` table with a clause reference.
2. Do not claim real SBI, HDFC or any bank integration. The bank is a mock/simulated adapter. Label it everywhere.
3. A PDF upload never proves authenticity. Only a bank confirmation (simulated here) or a recorded manual verification does.
4. Never label a document fraudulent from an automated flag. A flag creates a "manual review required" item with evidence for a human.
5. Never auto-release funds or auto-close a deposit on maturity. Maturity is not permission to release. Release needs checks and authorised human approval. The portal records requests and decisions; it does not move money.
6. If the bank source is down or slow, show "source unavailable / pending", retry with backoff, and route to manual verification. Never silently mark verified.
7. Use synthetic or anonymised data only. No real institution data in the repo.

8. No blockchain, no deep-learning forgery detection, no black-box AI decisions.
9. Every state change writes an audit entry in the same database transaction.
10. Release approval needs two different people (maker and checker).

6. Users and roles

- Institution / Trust user: submits FDR details, uploads evidence, sees status, answers deficiencies, requests renewal or release where permitted. Sees only their own institutions.
- AICTE officer (maker): reviews queues, checks evidence, records observations, raises release recommendations.
- AICTE senior / competent authority (checker): approves or rejects release; must be a different person from the maker.
- Bank officer (SIMULATED console): confirms or rejects FDR details and lien; a second bank user acts as checker.
- Auditor (read-only): sees history, audit log, evidence packs.
- System scheduler: a non-human actor that creates alerts and records events.
- Admin: manages users, roles and the rules table.

Access is role-based plus institution-scoped. Every endpoint checks both.

7. Domain model (core entities)

- Trust/Society (sponsoring body). One Trust can relate to many Institutions.
- Institution (id, name, trust_id, category, contacts).
- Instrument (id, institution_id, type [bank_fdr | online_deposit], bank, reference_no, amount_paise, issue_date, maturity_date, tenure_years, status, predecessor_id, rule_set_id). `predecessor_id` builds the renewal chain.
- FieldConfirmation (instrument_id, field, submitted_value, confirmed_value, source [bank_console | bank_file | manual], confirmed_by, confirmed_at, result [match | mismatch | pending | source_unavailable | manual_review]).
- Document (id, instrument_id, version, sha256, storage_key, uploaded_by, uploaded_at). Never overwritten.
- Alert (instrument_id, stage, channel, due_at, sent_at, acknowledged_at). UNIQUE (instrument_id, stage, channel).
- ReleaseRequest (instrument_id, kind [release | refund | closure], checklist, maker_id, checker_id, decision, reason, decided_at, bank_confirmed_at).
- Observation / ScrutinyNote (instrument_id, author, text, created_at).
- AuditEntry (seq, actor_id, role, entity, entity_id, from_state, to_state, payload_hash, prev_hash, row_hash, reason, at).
- Rule (key, value, clause_reference, effective_from). Holds thresholds, escalation days, tenure options, penalty percentages.
- Dispute / Hold (instrument_id, reason, opened_by, opened_at, cleared_at).

Money is stored as integer paise. Dates are stored as dates (not strings). Times are UTC, displayed in IST.

8. State machine

States: SUBMITTED, BANK_CHECK, ACTIVE, RELEASE_REQUESTED, COMPLIANCE_CHECK, APPROVED, BANK_RELEASE_PENDING, CLOSED. Exception states: REJECTED, RENEWAL_PENDING, DISPUTED.

Implement as a pure function `transition(state, event, context) -> (new_state, effects)` with a single transition table. No state change anywhere else.

From	Event	To	Actor	Guard
SUBMITTED	send_to_bank	BANK_CHECK	system	required fields and a document present
BANK_CHECK	bank_confirms	ACTIVE	bank checker	bank checker differs from bank maker; fields and lien confirmed
BANK_CHECK	bank_rejects	REJECTED	bank checker	reason recorded
REJECTED	resubmit	SUBMITTED	institution	corrected data
ACTIVE	renewal_detected	RENEWAL_PENDING	system	new reference seen in bank file or reported by institution
RENEWAL_PENDING	send_to_bank	BANK_CHECK	system	successor linked via predecessor_id; bank must re-confirm
ACTIVE	dispute_opened	DISPUTED	AICTE officer	reason recorded
DISPUTED	dispute_cleared	ACTIVE	senior	reason recorded
ACTIVE	release_requested	RELEASE_REQUESTED	institution or officer	no open dispute
RELEASE_REQUESTED	start_review	COMPLIANCE_CHECK	AICTE officer	checklist created
COMPLIANCE_CHECK	approve	APPROVED	senior (checker)	all clearances done; no open dispute; checker differs from maker
APPROVED	instruct_bank	BANK_RELEASE_PENDING	system	approval signed or

From	Event	To	Actor	Guard
				recorded
BANK_RELEASE_PENDING	bank_confirms_release	CLOSED	bank checker	bank confirmation recorded

Anything not in the table is rejected. Add tests for every row and for every forbidden transition.

Alert stage (for example T-180) is a label on the instrument, not a state.

9. Verification flow

1. Institution submits details and uploads evidence (hash stored).
2. System validates fields locally (format, dates, amount, tenure from rules).
3. Bank adapter returns one of: `match`, `mismatch` (with field and reason code), `pending`, `source_unavailable`, `manual_review`.
4. The UI shows two columns for each key field: Submitted and Bank-confirmed, with a status badge.
5. Reason codes: AMOUNT_MISMATCH, MATURITY_MISMATCH, LIEN_MISSING, CLOSED_AT_BANK, NOT_FOUND, HOLDER_MISMATCH.
6. A renewal seen in the bank file never auto-accepts. It moves the instrument to RENEWAL_PENDING and needs fresh bank confirmation.

Bank adapter interface (keep it small and stable): `verify(instrument_ref) -> VerificationResult`, `fetch_daily_feed(date) -> list[FeedRecord]`, `request_release(instrument_ref, release_id) -> Ack`. Implementations: `MockBankAdapter` (controlled responses, simulated maker-checker console), `FileFeedAdapter` (CSV/JSON drop). A real nodal-bank API adapter is future work.

10. Scheduler and alerts

- One idempotent daily job. It picks the largest reminder threshold already crossed (a missed run still alerts the next day).
- Starting thresholds (configurable): 180, 90, 60, 30 and 7 days before maturity, on the day, and overdue.
- Idempotency key: UNIQUE (instrument_id, stage, channel). The alert row and an outbox row commit in one transaction.
- Escalation ladder for unacknowledged alerts (configurable): institution after 1 day, officer after 7, senior after 15, head and auditor after 30.
- Nightly reconciliation compares portal records to the bank feed and creates review items, not automatic status changes.
- Provide a DEMO CLOCK (settable "now") so we can fast-forward time on stage. All date logic reads time from one `Clock` service.
- Run the scheduler as a separate worker process with a lock (advisory lock or `SELECT ... FOR UPDATE SKIP LOCKED`) so two instances never double-fire.

11. Release workflow

Request, scrutiny, parallel clearance checklist, approval by a checker who differs from the maker, bank release confirmation, then CLOSED. Block while any dispute or pending complaint flag is open. Placeholder clearance items until the handbook is checked: complaints, faculty/salary compliance, regional office NOC. Record reasons and timestamps for every step. Allow an evidence pack export (case history, document hashes, audit entries) as a PDF or JSON file.

12. Audit log

Append-only table. Each row: `row_hash = SHA-256(prev_hash || canonical_json(entry))`. No UPDATE or DELETE permission for the app role. Provide an integrity check endpoint and CLI that re-verifies the chain and reports the first broken row. Export the head hash daily to object storage. This gives tamper evidence, not prevention. Do not call it a blockchain.

13. Architecture and scalability

Build a modular monolith now that can scale out and be split later.

Modules (each owns its tables and exposes a service interface; no cross-module table access): `identity`, `institutions`, `instruments` (with lineage), `documents`, `verification`, `workflow` (state machine), `alerts` (scheduler and notifications), `release`, `reports`, `audit`, `rules`, `adapters`.

Scale-readiness rules (design for them; do not over-build):

- API instances are stateless. Sessions use JWT or an OIDC token. Run several instances behind a load balancer.
- PostgreSQL is the system of record. Add indexes on (institution_id, status), (maturity_date) and a partial index for active instruments. Use pagination on every list endpoint. Plan a read replica for dashboards.
- Audit table is partition-ready (by month) and append-only.
- Documents go to S3-compatible object storage (MinIO locally), versioned, hash recorded in the database.
- Notifications use the transactional outbox pattern with a worker, so email/SMS failures never block a transaction.
- Mutating endpoints accept an idempotency key.
- Adapters are behind interfaces, so the bank connection can change without touching core code.
- Everything configurable lives in the `rules` table or environment variables, not in code.
- Add structured logs with a request ID, health checks, and basic metrics from the start.
- Multi-institution by design: every query is scoped by institution and role.

14. Tech stack

Default (from the blueprint): React + TypeScript + Tailwind frontend; Python + FastAPI backend (modular monolith); PostgreSQL; MinIO/S3 storage; APScheduler or Celery Beat for the daily job; Keycloak (OIDC) for identity; Docker Compose for local; GitHub Actions CI.

NOTE: the team's plain-English explainer used Node.js/Express and node-cron only as examples. The stack is not final. If the repository already uses a different stack, follow the repository and tell me about the conflict. Keep the same module boundaries, data model and state machine in any stack.

Frontend: accessibility built to GIGW 3.0 / WCAG 2.1 guidelines (labels, contrast, keyboard use). Do not claim compliance.

15. Security

TLS in transit; AES-256 at rest for sensitive fields; role-based and institution-scoped access on every endpoint; MFA for officer roles where Keycloak allows; input validation; file type and size limits on uploads; virus scanning hook (stub is fine); secrets from environment only; dependency and container scans in CI. Design with the DPDP Act in mind (minimise and mask personal data). Do not claim DPDP compliance.

16. Demo and tests

Seed 20 to 50 synthetic institutions with instruments in varied states. The six scenarios from the problem statement must run end to end:

1. New FDR submission.
2. Discrepancy identification (maturity date mismatch shown red).
3. Validity tracking (bank confirms; instrument becomes ACTIVE).
4. Approaching maturity (demo clock advances; each alert fires once; escalation if unacknowledged).
5. Renewal (new reference seen; RENEWAL_PENDING; lineage link; bank re-confirms).
6. Authorised release (dispute blocks, then clears; maker and checker approve; bank confirms; CLOSED; evidence pack exported).

Tests: unit tests for the transition table, the alert-stage calculator and the hash chain; integration tests for each scenario; a test that editing an audit row makes the integrity check fail; a test that two scheduler runs never double-send.

Impact panel: measure our own before/after on synthetic data (record accuracy, verification turnaround, renewal timeliness, effort to answer "which FDRs mature in 90 days"). Only show numbers we measured. Label them "synthetic data".

17. Build order

1. Repo, Docker Compose, CI, config and rules table, Clock service.
2. Identity and roles; institutions, trusts, instruments; document upload with hash; audit log with hash chain and integrity check.
3. Bank adapter interface, mock bank console, file feed; field confirmation and two-column view; reconciliation outcomes.
4. Alert stages, escalation, outbox, demo clock; renewal lineage; release workflow with guards; dashboard and reports.
5. Seed data, six scenario scripts, impact panel, accessibility and security scans, rehearsal.

Do not start a phase before the previous gate works: rules confirmed, audit check passes, mismatch flagged end to end, six scenarios run.

18. Open questions (ask before assuming)

- Which handbook edition and deposit table apply to the demo data?
- What are AICTE's actual release conditions and their order?
- Does AICTE's portal expose readable data, or do we mock it?
- Final backend stack (Python/FastAPI vs Node/Express) and the final product name.
- Hackathon dates and demo length.

19. Repo layout (suggested)

```
/docs                # the PDFs and this file
/backend

/app/modules/{identity,institutions,instruments,documents,verification,workflow,alerts
  /app/core          # config, clock, db, security, logging
  /tests
/frontend
/infra               # docker-compose, CI, seed scripts
```

20. How to answer me

- Start with a one-line plan, then the code.
- Point out any conflict with this file instead of silently working around it.
- When you add a rule, threshold or clause, say where it must be confirmed.
- Keep commit-sized changes. Add or update tests with every change to the state machine, alerts or audit log.